



Agile Software Engineering

Week 1 – Introduction to agile methods

My background



Carsten Ruseng Jakobsen

M.Sc.E.E., EBA, PMP, CSP

Master from Ålborg Universitet in 1990

First job in Silicon Valley

Worked as developer, consultant, project and product manager

Written, published and presented articles with inventor of Scrum

Since 2017 running my own consulting business

34 years of experience from the industry as software developer and manager

Exam

Course learning goals:

- ☐ An introduction to agile software development, its principles, values and practices
- ☐ Provide insight into leading paradigms of agile software engineering and agile processes

Topics include but not limited to:

- ☐ Scrum, XP, Kanban
- ☐ Risk Management
- ☐ Quality Management
- ☐ Test

Exam

Exam:

Oral exam 20 mins

Select a topic from the course curriculum and present it

What is a software process

What is a software process?

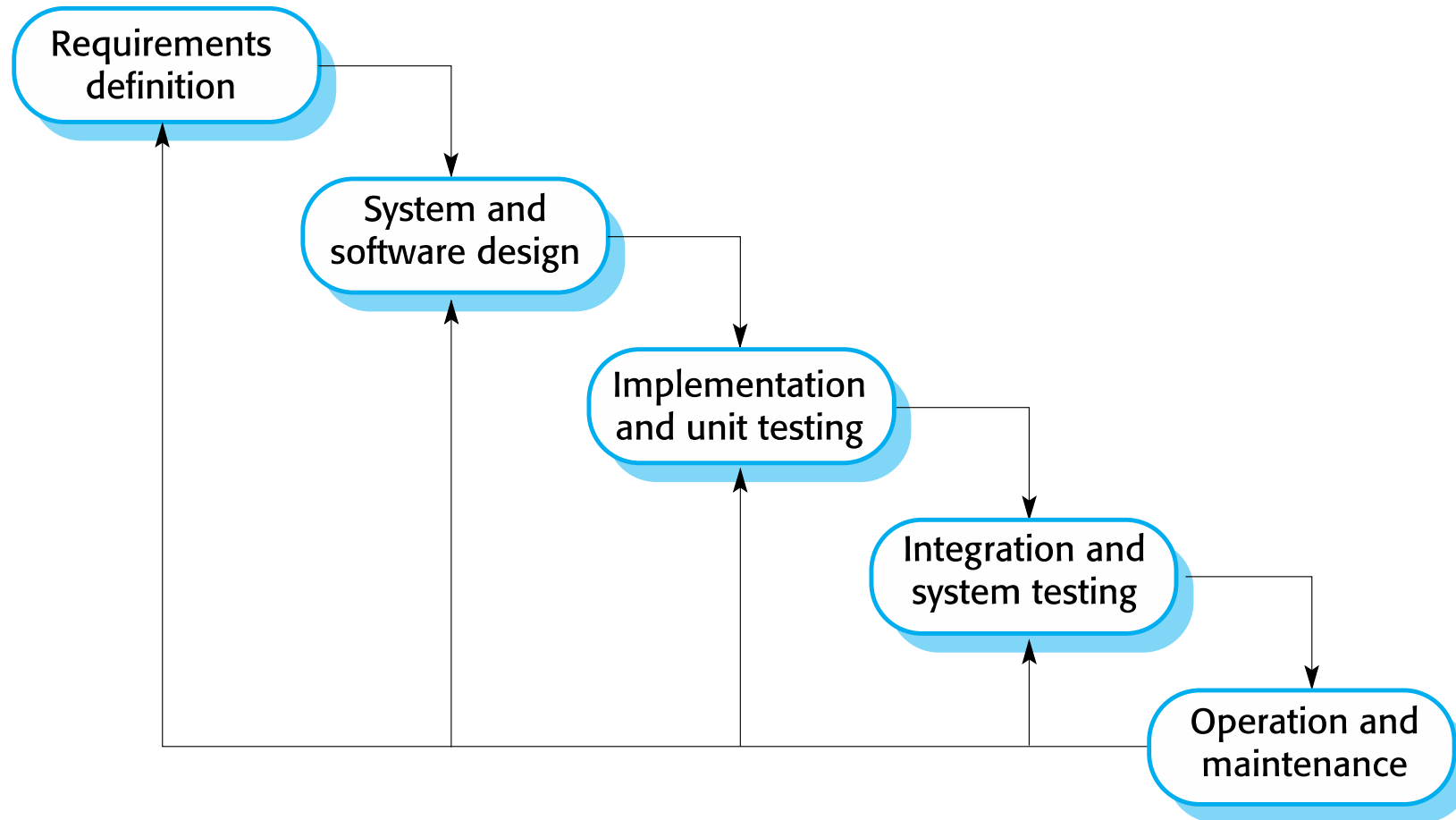
“a structured set of related activities required to develop a software system” (Sommerville chapter 2)

Software process activities

- ❑ **Specification** – defining what the system should do. During this activity, customer s and software development team defines the properties and requirements of the software product or service
- ❑ **Design and implementation** – defining the organization of the system and implementing the system.
- ❑ **Validation & verification** – checking that it does what the customer wants, “aka fit for purpose”. During this phase, the software is checked to ensure the product/service conforms to customers expectations and experience (validation) and specification (verification) through integration and system testing
- ❑ **Evolution** – changing the system in response to changing customer needs. The software is modified to reflect changing customer requirements

Plan-driven processes

Waterfall



Brainstorming



In a group of 3 discuss: (5 mins)

What are the drawbacks of the waterfall model approach to software development or product development in general?

Plan-driven processes

Waterfall drawbacks:

☐ Ridity

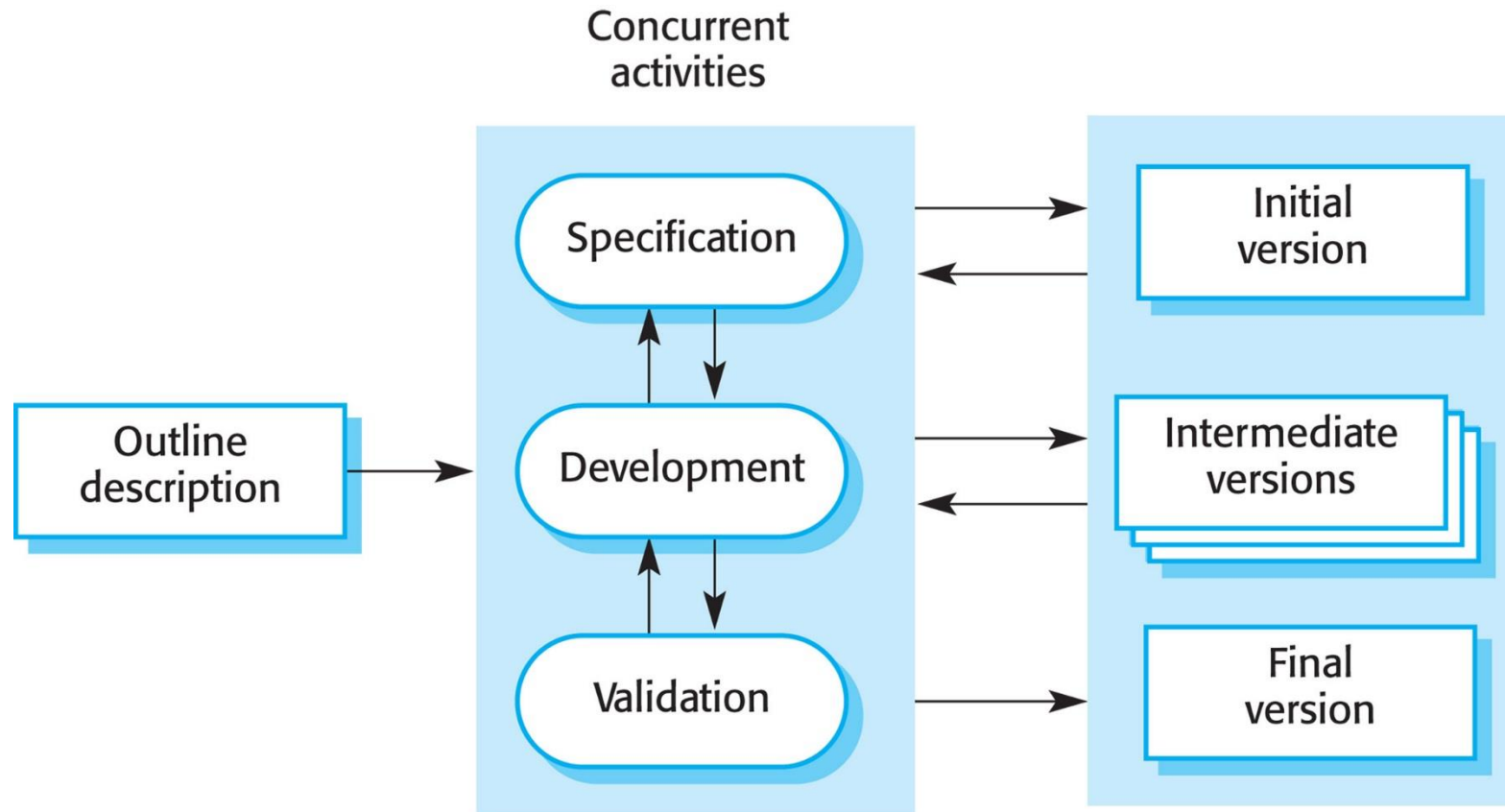
- ☐ Does not accommodate changing requirements. It assumes the business needs are fixed and everything is known upfront
- ☐ The final product is not visible to the customer until the very end. This is risky, the original assumptions made by the customer cannot be challenged
- ☐ Not suitable for projects with high level of uncertainty about the requirements, e.g., we do not know everything at the start
- ☐ The roll out to production is risky when the impact on the end users is high, i.e., “big-bang” approach

Waterfall advantages:

- ☐ Suitable for projects with stable requirements
- ☐ Stages are clearly defined and so are the deliverables

Plan-driven processes

Incremental development



Plan-driven processes

Incremental development:

- ❑ Incremental development is based on the idea of developing an initial implementation, then, seeking user feedback, and evolving it through several versions until an acceptable system has been developed.
- ❑ The activities of a process are not separated but interleaved with feedback involved across those activities.
- ❑ Each system increment is to develop a particular functionality of the software.
- ❑ Increments allow the customer to evaluate the software at early stage in the development
- ❑ It is plan-driven because the increment are defined in advance and only the current increment can be changed.

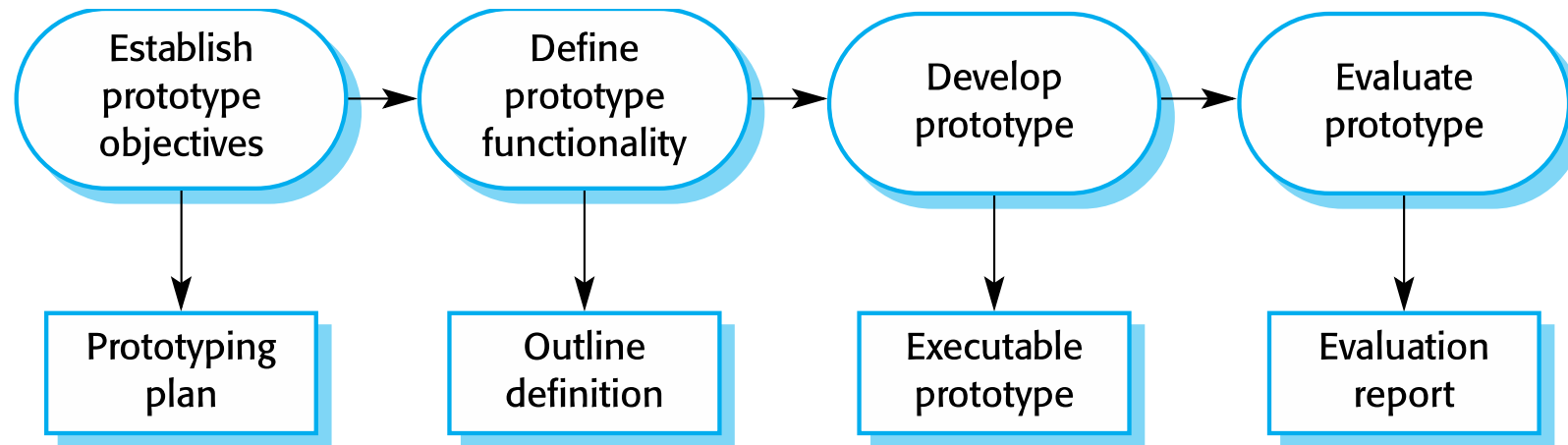
Plan-driven processes

Incremental drawbacks:

- ☐ Not suitable when the requirements are stable (e.g., regulations)
- ☐ Difficult to propose a design when increments are bringing changes to the software architecture. This may lead to degrading of the design
- ☐ The software may end up not scalable and future changes difficult to implement
- ☐ Refactoring and subsequently additional cost to to improve the software design

Plan-driven processes

Prototyping



Plan-driven processes

Prototyping:

- ❑ A prototype is a version of the whole software or part of it that's developed quickly to validate the customer's requirements and their feasibility. This allows the customer to see an early version of the software to test their assumptions and the development team to align their understanding of the requirements with the end customer.
- ❑ Prototyping is suitable when the requirements are novel and greenfield.
- ❑ The customer is involved during the development of the prototype, which increases the likelihood of accepting of the final product

Plan-driven processes

Prototyping drawbacks:

- ☐ Not a standalone process. It must be used within a waterfall of incremental development.
- ☐ Prototype are developed fast and usually their quality is sub-standards and they can develop to be come the final product

Plan-driven processes

The dissatisfaction with plan-driven approach:

☐ **Coping with change**

- ☐ Plan-driven processes are unable to cope with changing requirements. They have a low tolerance for change
- ☐ Business requirements are not always known at the outset of projects
- ☐ Business requirements are simply subject to change due to external factors and shifting business needs and priorities

☐ **Adding business value late**

- ☐ In large projects, the whole bulk of the value is delivered to the customer at the very end of the project. Large and complex projects take longer to deliver, hence the customer does not benefit from the value of the product sooner

☐ **Lacking manoeuvrability**

- ☐ Significant overhead in planning, designing and documentation. This can be justified for a large project, but is an overkill for small and medium projects

Brainstorming



In a group of 3 discuss: (5 mins)

How do you propose to reduce or solve some of the problems of plan-driven approaches to software development or product development?

What do you know about agile?

The Agile Manifesto



The Agile Manifesto

A painting of a group of people in a meeting, with text overlaid. The painting shows several people in a room, some standing and some sitting, looking at a screen or whiteboard. The style is somewhat impressionistic, with soft colors and visible brushstrokes. The text is overlaid in the center of the image.

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

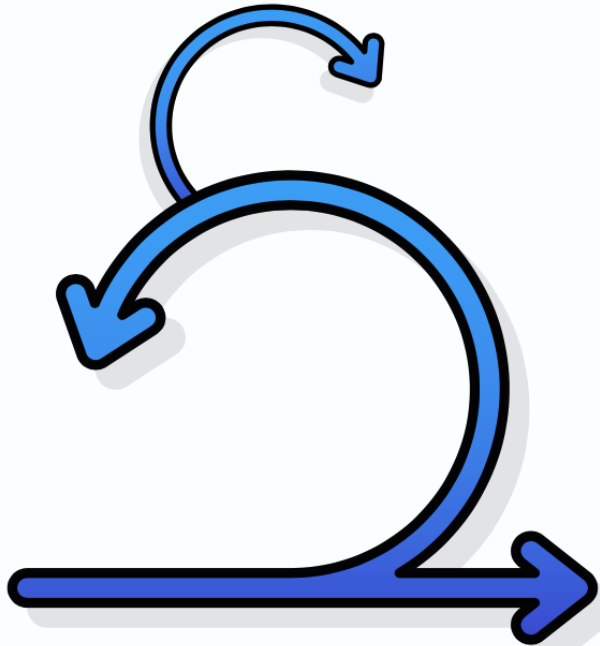
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

The rise of agile

AGILE

Iterative approach to software development



VS

WATERFALL

Plan-driven approach to software development

